

Frontend React para Fitbit API (Brainstorming, Plan y Prompts)

Autor: React Frontend Assistant (rol: ingeniero de software / diseñador front-end)

Resumen ejecutivo

Este documento contiene un plan detallado y accionable para crear el frontend cliente en React para la API Laravel (Fitbit API) que has provisto. Incluye: compatibilidad entre frameworks, recomendaciones sobre contenedores (Docker), selección de modelos de IA para generar código (vibe-coding), cómo mapear la documentación Swagger a acciones frontend, un **to-do list** de planificación, prompts (system prompt + ASPECCT) para orquestar la generación automatizada del frontend y ejemplos prácticos.

Nota: este plan asume que la API provista expone endpoints con autenticación Bearer (Laravel Passport) y documentación OpenAPI/Swagger funcional (según los archivos que proporcionaste).

0. Fuentes de contexto

- README proporcionado (contexto del proyecto, endpoints, setup, testing).
- Documentación Swagger completa del API (OpenAPI 3.0, esquemas, ejemplos de request/response).

(Usaste README y Swagger para el análisis del backend y la documentación interactiva.)

1. Manejar la compatibilidad entre frameworks (frontend React ↔ backend Laravel)

Principios

1. **Contrato via OpenAPI:** La especificación OpenAPI (Swagger) es la verdad. Generar un cliente TypeScript/JS a partir del `api-docs.json` evita errores manuales y mantiene contrato entre frontend y backend.
2. **Autenticación:** Laravel Passport usa tokens Bearer. El frontend debe implementar: login -> recibir token -> guardarlo (preferentemente en memoria + cookie httpOnly o secure storage con atención a XSS), añadir interceptor `Authorization: Bearer {token}` y gestionar expiración/refresh según política (en README indican tokens de 15 días; si no hay refresh token, renovar con login).
3. **Errores y status codes:** Mapear 401/403/422 al UI: redirección a login, mostrar modal de permisos, formularios con validaciones.
4. **Versiónado de API:** Usa prefijo `/api/v1/...` desde spec. Mantén una capa de `api/services` que encapsule llamadas para facilitar cambios futuros.

5. **CORS / CSRF:** Configurar CORS en Laravel (ya debería) y en la app React hacer peticiones desde origen correcto. Para producción, configurar `APP_URL` y `APP_ENV` en backend y `VITE_API_URL` en frontend.

Arquitectura recomendada frontend

- **Lenguaje:** TypeScript (mejor tipado con spec OpenAPI)
- **Bundler:** Vite (rápido, moderno)
- **Routing:** React Router v6
- **Fetching & Cache:** TanStack Query (react-query) o RTK Query
- **State global:** Context + hooks para Auth (o Zustand si prefieres) — evitar Redux salvo necesidad compleja
- **Component library:** TailwindCSS (rápido) + librería de componentes (Headless UI / shadcn) si quieres UI consistente
- **Tests:** Vitest + React Testing Library

2. Docker u otra herramienta para evitar colisiones de versiones

¿Por qué usar Docker?

- Asegura entorno idéntico entre desarrollador/ci/producción.
- Evita conflictos de versiones Node/Python/PHP/DB locales.

Propuesta práctica

- **Docker para backend:** No obligatorio si backend ya corre en XAMPP local, pero recomendable si quieres reproducibilidad y CI.
- **Docker para frontend:** Opcional; usar `node:20-alpine` para build. Usar docker-compose para orquestar frontend, backend y db local.

Ejemplo de enfoque (docker-compose)

- Servicios: `frontend` (React+Vite), `backend` (PHP-FPM + Nginx o imagen del proyecto Laravel), `db` (mysql:8), `adminer`.
- Volúmenes: código fuente montado en frontend/backend para desarrollo.

Beneficios: aislar versiones de Node/NPM, reproducible CI -> puedes usar las mismas imágenes en pipelines.

Alternativas

- **nvm / volta** para manejar versiones Node localmente
- **devcontainers (VS Code)** para reproducir entorno sin Docker en todo el equipo

Recomendación: usar `docker -compose` para el equipo y `devcontainer` para onboarding rápido.

3. Selección de modelos de IA para “vibe coding” (generación de frontend)

Buscamos modelos/servicios que sean buenos generando código, entendiendo especificaciones OpenAPI, y que permitan iteración: debug, tests unitarios, y producción de componentes React+TypeScript.

Candidatos y rol para cada uno

1. OpenAI (GPT-4o / GPT-4o-mini / GPT-4o-code, si disponible)

2. Pros: excelente para generación de componentes, tests, y prompts complejos. Buen soporte de few-shot.

3. Contras: coste, límites de uso; necesitas controlar seguridad y revisar salida.

4. Anthropic Claude (Claude 3 Haiku / Pro)

5. Pros: fuerte en instrucciones largas y seguridad; buen razonamiento.

6. Contras: coste y latencia variable.

7. Gemini / Google (si disponible públicamente)

8. Pros: buenas capacidades de generación y contexto largo; integración con infra Google.

9. Meta / Llama 2 / Llama 3 (via Hugging Face o infraestructura privada)

10. Pros: opción on-prem / fine-tune; menor coste si self-host.

11. Contras: menor desempeño out-of-the-box en prompts complejos comparado con top-tier closed models.

12. Mistral / Mistral-Instruct

13. Pros: buenos para código, coste competitivo.

14. Código-specific assistants: GitHub Copilot / Replit AI / Codeium

15. Pros: están optimizados para asistente de codificación en editores; excelente para generar snippets, completar funciones y tests.

Recomendación práctica (combinada)

- **Primario (generación de alto nivel, diseño y scaffolding):** GPT-4o (o Claude) por su capacidad de entender spec y generar arquitecturas y componentes.
- **Secundario (iteración rápida en editor):** GitHub Copilot / Replit AI para autocompletar y producir componentes de forma interactiva.
- **On-prem / privacidad:** si datos sensibles, usar Llama3/Mistral en infra propia para generación offline.

Motivación: usar un modelo grande para orquestar y diseñar (explicar, mapear endpoints), y copilots para producir código día-a-día.

4. Entendimiento de la documentación provista y ajuste a las funciones del frontend

Lo que entendí (basado en README y Swagger):

- API con endpoints versionados `api/v1/*` para Auth, Users, Disciplines, Communities y Stats. filecite turn0file0 filecite turn0file1
- Seguridad: autenticación Bearer con Laravel Passport; tokens de 15 días (no se mencionan refresh tokens explícitos). filecite turn0file0
- Endpoints públicos: listar disciplinas/communities; endpoints protegidos: crear/editar/eliminar según roles.
- OpenAPI disponible: `api-docs.json` y `api-docs.yaml` — ideal para generar cliente TypeScript. filecite turn0file1

Implicaciones para frontend

1. **Generar Cliente API TypeScript:** usar `openapi-generator-cli` o `swagger-typescript-api` para producir modelos y servicios. Esto acelera la integración.
2. **Flujo Auth:** implementar hook `useAuth()` y `AuthProvider` que gestione login, logout, `me` endpoint y almacenamiento del token.
3. **Roles y UI:** renderear botones/acciones según `role` del `me` response (User/Admin/Moderator).
4. **Paginación / Filtros:** utilizar query params desde spec; un componente `List` reutilizable.
5. **Estadísticas:** endpoints de stats se mapearán a dashboards (gráficos, tablas). Usar Recharts o Chart.js con datos agregados.

Si algo no está cubierto por la spec (por ejemplo, refresh tokens), recomiendo: - Pregunta al backend si existe refresh token. Si no existe, implementar re-login o endpoint custom para refresh.

5. ¿Cómo hacer que el modelo entienda la compatibilidad backend/frontend?

Estrategia de prompt y data inputs

1. **Proveer el OpenAPI JSON (o YAML) como input:** el primer paso es pasar la spec al modelo (o una URL) y pedirle que la analice; así el modelo no adivina endpoints.
2. **Proveer el README / notas de implementación** (ya hecho) para reglas adicionales (roles, expiración tokens, ejemplos de requests).
3. **Pedir outputs estructurados:** generación de `client.ts`, ejemplos de `fetch` / `axios`, y tests unitarios para endpoints claves.

Limitaciones prácticas

- Modelos generalmente no ejecutan la spec; conviene usar herramientas automatizadas (OpenAPI Generator) y pedir al modelo que genere wrappers y UI usando esos artefactos.

6. TODO-list (plan de trabajo) — entregable mínimo viable (MVP)

Fase 0 — Preparación

- [] Revisar `api-docs.json` y `api-docs.yaml` (descargar / abrir). (desde `http://localhost:8000/docs/api-docs.json`). filecite turn0file1
- [] Decidir stack: React + Vite + TypeScript + Tailwind + React Router + TanStack Query.
- [] Crear repo `fitbit_frontend` y configurar CI básica (GitHub Actions: build/test).

Fase 1 — Scaffolding y Cliente API

- [] Inicializar proyecto Vite + TS
- [] Añadir ESLint, Prettier, Husky, lint-staged
- [] Generar cliente TypeScript con OpenAPI Generator (`typescript-axios` o `typescript-fetch`) apuntando al `api-docs.json` local
- [] Crear carpeta `src/api/` con cliente generado

Fase 2 — Auth y navegación básica

- [] Implementar `AuthProvider` + `useAuth()` (login, logout, me)
- [] Crear `Login` y `Register` pages usando endpoints `/api/v1/login` y `/api/v1/register`
- [] Proteger rutas con `RequireAuth` HOC/component

Fase 3 — CRUD principal

- [] Implementar `Disciplines` list + detail + admin form (create/update/delete)
- [] Implementar `Communities` list + detail + create/join flows
- [] Implementar `Users` list/profile (role-based)

Fase 4 — Dashboard & Stats

- [] Implementar `CommunityStats` dashboard (ranking, percentage, summary)
- [] Visualizaciones con Recharts

Fase 5 — Tests y QA

- [] Tests unitarios para components críticos
- [] E2E tests (Playwright) para flows auth + community CRUD

Fase 6 — Contenedores y despliegue

- [] Añadir `Dockerfile` para frontend y `docker-compose.dev.yml` para integración con backend
- [] Configurar CI para builds y despliegue estático (Netlify/Vercel/Cloudflare Pages)

7. System prompt, método ASPECCT y prompt engineering (para ejecutar en el modelo)

Propuesta: ASPECCT — acrónimo para diseñar prompts reproducibles

- **Authentic Context:** incluir README + OpenAPI spec + reglas de auth (token duration, roles)
- **Scope:** delimitar exactamente qué generar (ej. `Login` + `AuthProvider` + `useAuth` + tests)
- **Prompt:** el propio texto instructivo con constraints (lenguaje, estilo, estructura)
- **Examples:** incluir 1-2 ejemplos de entrada/salida (few-shot)
- **Constraints:** tamaño del archivo, dependencias permitidas (axios, tanstack-query), no usar class components
- **Checklist:** items que el modelo debe devolver (component TSX, hooks, tests, README de uso)
- **Test:** pedir que genere tests unitarios y casos de uso demostrables

System prompt (para el modelo que generará código)

Eres un asistente especializado en generar frontend React + TypeScript de alta calidad. Tu objetivo es producir código listo para integrar con una API Laravel documentada por OpenAPI 3.0. Lee el OpenAPI JSON proporcionado y genera: 1) cliente TypeScript (si no existe), 2) AuthProvider y hook `useAuth`, 3) pantallas `Login` y `Register`, 4) example lists para `Disciplines` y `Communities`, 5) tests unitarios para `Auth` y `Disciplines`. Cumple con las siguientes restricciones: usa Vite, TypeScript, React 18+, React Router v6, TanStack Query para fetching, axios (o fetch) como cliente HTTP; cadencia de token: Bearer. No uses class components. Devuelve archivos separados (ruta y contenido). Comenta brevemente cada archivo. Si el spec contiene campos extra, mapea model objects con tipos TypeScript.

Ejemplo de prompt de ejecución (few-shot minimal)

Input: `api-docs.json` (attach or URL), `README.md` (context),
`target=Login+AuthProvider+useAuth`

Prompt:

Usa el OpenAPI JSON adjunto. Genera en TypeScript (Vite+React) los siguientes archivos:

- `src/api/client.ts` (axios instancia con `baseUrl = process.env.VITE_API_URL`)
- `src/contexts/AuthContext.tsx` (AuthProvider que expone `user`, `token`, `login`, `logout`)
- `src/hooks/useAuth.ts` (hook)
- `src/pages/Login.tsx` (formulario con validaciones mínimas)
- `tests/Auth.test.tsx` (Vitest + React Testing Library)

Requisitos:

- Token guardado en memoria y en cookie secure (ejemplo). Incluye manejo de 401 global (interceptor axios).
- Cada archivo: comentar su propósito y endpoints usados.
- Devuelve el contenido de cada archivo en JSON con campos `{path, content}`.

Prompt para generar componentes de CRUD a partir de un endpoint

Dada la ruta `GET /api/v1/disciplines` (ver esquema en spec), genera:

- src/pages/Disciplines/List.tsx (listado con paginación básica y link a detail)
- src/pages/Disciplines/Detail.tsx
- src/components/DisciplineForm.tsx (create/update)
- tests/Disciplines.test.tsx

Usa TanStack Query para fetching y query invalidation tras create/update/delete.

Control de calidad automático (post-generation)

- Pedir al modelo generar tests unitarios y un script `lint:fix && vitest`.
- Pedir un checklist de revisión manual (seguridad, XSS, CSRF, token storage).

8. Pipelines y automatización recomendada

- **CI:** GitHub Actions que ejecute `npm ci`, `npm run lint`, `npm run test`, `npm run build`.
- **CD:** Deploy a Vercel/Netlify o build estático servido por Nginx dentro de Docker.
- **IaC:** si despliegas backend y frontend juntos, usar infra (Terraform) o AWS S3+CloudFront para frontend.

9. Seguridad y consideraciones finales

- **Token storage:** preferir cookies `httpOnly` si backend puede crear cookie; si no, guardar token en memoria + refresh strategy para minimizar XSS risk.
- **HTTPS** obligatorio en producción.
- **Rate limiting / retry policy** en cliente.
- **Validación de inputs** en frontend, pero confiar en validaciones del backend.

10. Entregables sugeridos para la primera entrega (MVP)

1. Repo `fitbit_frontend` con Vite+TS config
2. Cliente generado desde OpenAPI (typescript-axios)
3. Auth flow (Login/Register/ProtectedRoute)
4. Pages básicas: Disciplines list/detail, Communities list
5. Dashboard minimal con `stats/communities/summary`
6. Dockerfile + docker-compose.dev.yml
7. Tests unitarios básicos y CI pipeline

Anexos: comandos y snippets útiles

- Generar cliente TS (OpenAPI Generator):

```
openapi-generator-cli generate -i http://localhost:8000/docs/api-docs.json -g typescript-axios -o ./src/api/generated --additional-properties=supportsES6=true
```

- Vite init:

```
npm create vite@latest fitbit_frontend -- --template react-ts
cd fitbit_frontend
npm install
```

- Example axios instance (sugerido) - el modelo generador lo producirá en `src/api/client.ts`.

Observaciones finales

- El enfoque híbrido (modelo grande para orquestación + copilots para codificación diaria) ofrece velocidad y control.
- El contrato OpenAPI reduce ambigüedad: prioriza generar cliente de API para evitar bugs de integración.
- Si quieres, puedo: 1) generar los prompts listos para ejecutar en un modelo (GPT/Claude), 2) crear los archivos scaffold (TSX, hooks, cliente) aquí mismo en el repo, o 3) producir los artefactos Docker y CI.

Fin del documento.